ECG-Net: Vision-to-Signal ECG Digitization

Piyush Panda

December 2025

Index

Contents

1 Introduction **6**

2 Problem Statement and Why the Formulation Matters **6**

2.1 Task definition 6

2.2 Why not direct image-to-waveform regression? 6

3 Data Unit, Splitting Logic, and Leakage Control **7**

3.1 Training unit 7

3.2 Why record-level splitting instead of panel-level splitting? 7

3.3 Configured split fraction 7

4 Input Representation and Preprocessing Pipeline **8**

4.1 Panel geometry 8

4.2 Base channels 8

4.3 Trace-enhancement channel 8

4.4 Mechanism of the trace-enhancement channel 8

4.5 Why grayscale first? 8

4.6 Black-hat morphology: what it does and why it helps 9

4.7 Canny edges: what they contribute 9

4.8 Gaussian smoothing: why not keep the map sharp? 9

4.9 Why engineer this channel at all? 9

5 Target Representation: Soft Gaussian Heatmap **9**

5.1 Definition 9

5.2 Why not use a hard binary line? 10

5.3 Why sigma equals 2.0? 10

6 Architecture Overview **10**

6.1 Core architectural idea 10

6.2 Why EfficientNet-B0? 11

6.3 Why a U-Net-like decoder? 11

6.4 Architecture flow, step by step 11

6.5 What is Conv-BN-SiLU? . . . 11
6.6 Why this block sequence fits ECG-Net . . . 12

7 Differentiable Waveform Extraction by Softargmax 12
7.1 The problem it solves . . . 12
7.2 Column-wise probability interpretation . . . 12
7.3 Expected row extraction . . . 12
7.4 Why not use hard argmax? . . . 12
7.5 Conversion from row index to millivolts . . . 13
7.6 Why temperature 10.0? . . . 13

8 Loss Design: Mathematical Form and Intuition 13
8.1 Why a composite loss is necessary . . . 13
8.2 Notation . . . 13
8.3 Heatmap loss: BCEWithLogits + Dice . . . 14
8.3.1 BCE term . . . 14
8.3.2 Dice term . . . 14
8.3.3 Combined image-space loss . . . 14
8.4 Waveform L1 loss . . . 14
8.5 Energy-aware weighting . . . 15
8.6 First-derivative smoothness term . . . 15
8.7 Second-derivative smoothness term . . . 15
8.8 Shift-tolerant alignment loss . . . 15
8.9 Total objective by phase . . . 16
8.10 Amateur-friendly summary of the losses . . . 16

9 Training Curriculum and Optimization Strategy 16
9.1 Two-phase training logic . . . 16
9.2 Optimizer and training hyperparameters . . . 17
9.3 Augmentation policy . . . 17

10 Metrics: What to Measure and Why 17
10.1 Primary metric: best-shift SNR . . . 17
10.2 Secondary metric: trace_acc@2px . . . 17
10.3 Why not optimize for Dice alone? . . . 18
10.4 Recommended operating bands . . . 18

11 How ECG-Net Differs from Standard OCR **18**

12 Alternatives and When They Can Outperform ECG-Net **18**

12.1 Direct image-to-waveform regression 18

12.2 Pure segmentation plus non-differentiable vectorization 19

12.3 Classical computer vision pipeline 19

12.4 Different backbones 19

12.5 Alternative losses 19

13 Senior-Level Architecture Defense **20**

13.1 Why record-level splitting is the only honest evaluation choice 20

13.2 Why the trace-enhancement channel is not a hack 20

13.3 Why the soft Gaussian heatmap is a probabilistic target 20

13.4 Why softargmax is the architectural hinge 20

13.5 Why checkpoint by SNR instead of raw loss 20

14 Interview Q&A Set for ECG-Net **20**

14.1 Q1. Why record-level splitting rather than panel-level splitting? 20

14.2 Q2. Why use a handcrafted trace-enhancement channel in a deep model? 21

14.3 Q3. Why a soft Gaussian heatmap instead of a hard line target? 21

14.4 Q4. What is Conv-BN-SiLU doing for you? 21

14.5 Q5. Why softargmax? 21

14.6 Q6. How is this different from OCR? 21

14.7 Q7. Why these losses? 21

14.8 Q8. When would direct waveform regression outperform this design? 21

14.9 Q9. Why not train with DTW directly? 21

14.10 Q10. What is the biggest limitation of the current system? 22

15 Compact Hyperparameter Summary **22**

16 Hyperparameter Roles and Why These Values Make Sense **22**

16.1 Geometry and representation hyperparameters 23

16.2 Trace-enhancement hyperparameters 23

16.3 Target and differentiable decoding hyperparameters 23

16.4 Class imbalance and reconstruction-loss hyperparameters 24

16.5 Smoothness and alignment hyperparameters 24

16.6 Optimization hyperparameters 25

16.7 Training-schedule hyperparameters 26
16.8 How to interpret these hyperparameters as a system 26

17 Final Technical Summary **26**

1 Introduction

ECG-Net is a hybrid vision-to-signal learning system for digitizing raster ECG reports into calibrated 1D waveforms. Instead of directly regressing a waveform vector from an input image, the system first predicts a dense trace heatmap, then decodes that heatmap into a differentiable waveform estimate, and finally optimizes both image-space and signal-space objectives together.

The central design belief is simple: for ECG images, the first problem is usually *where the trace is*, not directly *what the amplitude value is*. Therefore, ECG-Net solves localization first and numerical reconstruction second, but keeps both stages coupled during training.

At a high level, the learned path is:

$$\text{lead panel image} \rightarrow \text{trace heatmap} \rightarrow \text{softargmax waveform} \rightarrow \text{mV-domain supervision.}$$

This report rewrites the original project report template around ECG-Net and incorporates deeper explanations of the architectural choices, the preprocessing pipeline, the mathematical formulation, the rationale behind the losses, the main alternatives, and the senior-level interview questions that naturally arise from the design.

2 Problem Statement and Why the Formulation Matters

2.1 Task definition

Given a cropped ECG lead panel image, predict the underlying ECG waveform in calibrated physiological units.

Formally, for an input image panel $I \in \mathbb{R}^{C \times H \times W}$, ECG-Net predicts a 2D heatmap

$$\hat{H} \in [0,1]^{H \times W},$$

then decodes it into a 1D signal

$$\hat{s} \in \mathbb{R}^{W},$$

where each column corresponds to one horizontal sample location and each value is the predicted waveform amplitude in millivolts.

2.2 Why not direct image-to-waveform regression?

Direct regression is possible, but it is a weaker inductive bias for this problem. A scanned ECG sheet contains thin traces, grid lines, paper texture, rotation, blur, background clutter, and calibration artifacts. These are primarily *geometric* nuisances. If the model is asked to predict waveform values immediately, it must learn localization, denoising, calibration, and sequence regression all at once.

ECG-Net instead decomposes the task into a more natural sequence:

1. localize the trace as a visual object,
2. decode the localized trace into a signal,
3. supervise that decoded signal directly.

This decomposition makes the task easier to optimize and easier to debug.

3 Data Unit, Splitting Logic, and Leakage Control

3.1 Training unit

The training unit is one cropped lead panel, not one full ECG page. This is sensible because each lead panel corresponds to one local trace reconstruction problem with a fixed panel geometry.

Each sample returns:

- input tensor x,
- target heatmap y,
- target waveform s_{mv},
- per-sample pixels-per-mV calibration value ppmv.

3.2 Why record-level splitting instead of panel-level splitting?

This is one of the most important evaluation choices in the project.

Suppose a single ECG record generates multiple lead panels. Those panels share:

- the same scan conditions,
- the same page style and printing texture,
- the same grid characteristics,
- similar skew and crop artifacts,
- the same acquisition context.

If lead I from a record is placed in training and lead II from the same record is placed in validation, the validation set is no longer genuinely independent. The model has already seen record-specific appearance statistics. This produces leakage and inflates validation performance.

So ECG-Net uses **record-level splitting**, not panel-level splitting. Let $\mathcal{R}$ denote the set of records and let each record $r \in \mathcal{R}$ contain several panels. The split is performed over $\mathcal{R}$, not over individual panels. In words:

all panels from record r must belong entirely to either train or validation.

This preserves evaluation integrity and yields a more honest estimate of generalization.

3.3 Configured split fraction

The implementation uses:

$$\texttt{VALID_FRAC} = 0.10.$$

4 Input Representation and Preprocessing Pipeline

4.1 Panel geometry

Each panel is standardized to:

$$H = 256, \qquad W = 512.$$

This is not arbitrary. It makes the output heatmap geometry stable, keeps decoding column-aligned, and gives enough width to represent waveform morphology while keeping computation moderate.

4.2 Base channels

The default image input is RGB scaled to $[0, 1]$, so a naive input would be:

$$3 \times 256 \times 512.$$

4.3 Trace-enhancement channel

ECG-Net optionally adds a handcrafted fourth channel, so the effective input becomes:

$$4 \times 256 \times 512.$$

This extra channel is not another learned branch. It is an engineered cue map intended to amplify the ECG trace before the tensor reaches the encoder.

4.4 Mechanism of the trace-enhancement channel

The trace-enhancement channel is constructed from the following steps:

1. grayscale conversion,
2. black-hat morphology,
3. Canny edge extraction,
4. Gaussian smoothing,
5. weighted fusion.

The fusion weights are:

$$0.70 \times \text{black-hat} + 0.30 \times \text{edge map}.$$

The black-hat kernel scale is tied to image size using:

$$\texttt{TRACE_BH_FRAC} = \frac{1}{48}.$$

4.5 Why grayscale first?

ECG digitization is mostly about structure and contrast rather than semantic color. Converting to grayscale emphasizes intensity topology and makes morphology and edge extraction more stable.

4.6 Black-hat morphology: what it does and why it helps

Black-hat morphology is defined as:

$$\mathrm{BlackHat}(I) = \mathrm{Closing}(I) - I.$$

Here, closing is dilation followed by erosion. Intuitively, black-hat emphasizes small dark structures that sit below the local background. That is exactly the visual pattern of a printed ECG trace: a thin dark line on a lighter paper field.

This makes black-hat a strong operator for highlighting likely trace pixels while suppressing smoother low-frequency background variation.

4.7 Canny edges: what they contribute

Canny edge detection highlights strong local intensity transitions. For ECG-Net, it contributes information about the boundaries and local contour of the trace. However, edges alone are not enough, because gridlines and text can also generate strong edges.

That is why the fusion weights favor black-hat over Canny. The design assumption is that *trace-like dark foreground* is more reliable than *generic boundaries*. Hence the 0.70 / 0.30 weighting.

4.8 Gaussian smoothing: why not keep the map sharp?

The trace-enhancement channel is used as a soft prior, not as a hard mask. Smoothing reduces brittleness, suppresses pixel-level noise, and makes the auxiliary channel more compatible with the early convolutional filters.

4.9 Why engineer this channel at all?

This choice becomes even more sensible because the encoder is configured with:

$$\texttt{PRETRAINED_ENCODER} = \texttt{False}.$$

Without pretrained low-level visual priors, it is beneficial to inject domain-informed cues manually. The trace channel acts as a useful inductive bias for a thin-structure localization task.

5 Target Representation: Soft Gaussian Heatmap

5.1 Definition

The target is not a binary 1-pixel centerline. Instead, the waveform is projected into image space and a soft Gaussian band is drawn around the trace location.

For each column x, let the target row be $y^\star(x)$. The target heatmap is:

$$H(y, x) = \exp\left(-\frac{(y - y^\star(x))^2}{2\sigma^2}\right),$$

where the configured blur is:

$$\texttt{GAUSS_SIGMA_PX} = 2.0.$$

The full target shape is:

$$256 \times 512.$$

5.2 Why not use a hard binary line?

A hard line makes training brittle because:

- the foreground is extremely sparse,
- tiny alignment errors are punished too aggressively,
- scanned traces have real thickness and blur,
- gradients become too concentrated.

A Gaussian band solves all four issues. It says: the center row is best, but nearby rows are still plausible. That is a much better training signal for thin noisy curves.

5.3 Why sigma equals 2.0?

The choice of $\sigma = 2.0$ pixels is a compromise:

- too small: training becomes sharp, sparse, and unstable,
- too large: the target becomes blurry and permits imprecise localization,
- moderate value: gradients remain informative while the target stays curve-like.

At height 256, a 2-pixel standard deviation is narrow enough to preserve thin-structure precision but wide enough to tolerate scanning ambiguity.

6 Architecture Overview

6.1 Core architectural idea

The network is a U-Net-like encoder-decoder built around a `timm` feature extractor backbone. At a high level:

$$\begin{aligned}&\text{input panel} \to \text{multi-scale encoder features} \to \text{bottleneck refinement}\\&\to \text{skip-connected decoder} \to \text{1-channel heatmap logits.}\end{aligned}$$

The backbone is:

`tf_efficientnet_b0`.

The model uses `features_only=True`, which means the encoder is not used as a classifier; instead it emits a hierarchy of feature maps for U-Net-style decoding.

6.2 Why EfficientNet-B0?

EfficientNet-B0 gives a strong efficiency-to-representation trade-off. For ECG digitization, the goal is not giant semantic abstraction but compact multi-scale visual reasoning for thin curves. B0 is therefore a good practical backbone.

6.3 Why a U-Net-like decoder?

Thin-structure tasks require both:

- large enough receptive field to understand continuity and layout,
- high-resolution recovery to localize the curve precisely.

Skip connections help the decoder recover detail lost by downsampling while keeping semantic context from deep layers.

6.4 Architecture flow, step by step

Let the input tensor be x. The encoder produces multi-scale features:

$$(f_1, f_2, f_3, f_4) = \text{Encoder}(x).$$

Then the deepest feature map is refined in the bottleneck:

$$z = \text{Bottleneck}(f_4).$$

The decoder repeatedly upsamples, concatenates a skip, and refines:

$$d_3 = \text{Dec}_3(\text{Up}(z), f_3),$$

$$d_2 = \text{Dec}_2(\text{Up}(d_3), f_2),$$

$$d_1 = \text{Dec}_1(\text{Up}(d_2), f_1).$$

The final heatmap logits are emitted by a 1×1 convolution:

$$\hat{Z} = \text{Head}(d_1).$$

6.5 What is Conv-BN-SiLU?

Several blocks in the bottleneck and decoder use the pattern:

$$\text{Convolution} \rightarrow \text{BatchNorm} \rightarrow \text{SiLU}.$$

Convolution. This learns spatial filters that detect local structures such as edges, line fragments, bends, and motif patterns.

BatchNorm. This stabilizes feature magnitudes across batches and makes optimization easier.

SiLU. The SiLU activation is:

$$\mathrm{SiLU}(x) = x\,\sigma(x),$$

where $\sigma(x)$ is the sigmoid.

Compared with ReLU, SiLU is smoother and does not abruptly zero out everything negative. That smoothness is often useful in modern vision models where you want less brittle feature transitions.

6.6 Why this block sequence fits ECG-Net

ECG traces are continuous thin structures. A smooth nonlinear block stack such as Conv-BN-SiLU is a good fit for preserving subtle line evidence while still learning nonlinear feature transformations.

7 Differentiable Waveform Extraction by Softargmax

7.1 The problem it solves

A heatmap alone is not yet a waveform. ECG-Net must map column-wise spatial probabilities into a 1D signal while preserving differentiability.

7.2 Column-wise probability interpretation

Let the logits be $\hat{Z} \in \mathbb{R}^{B\times 1\times H\times W}$. For each column x, the model applies a softmax over the height dimension:

$$p(y \mid x) = \frac{\exp(T\hat{Z}(y,x))}{\sum_{u=1}^{H}\exp(T\hat{Z}(u,x))},$$

where the temperature is:

$$\texttt{SOFTARGMAX_TEMP} = 10.0.$$

7.3 Expected row extraction

Instead of taking a hard argmax, ECG-Net computes the expected row index:

$$\hat{y}_{\mathrm{soft}}(x) = \sum_{y=1}^{H} p(y \mid x)\, y.$$

This is a softargmax.

7.4 Why not use hard argmax?

Hard argmax is non-differentiable. If hard argmax were used inside training, the waveform loss would not backpropagate cleanly into the heatmap logits. Softargmax preserves the crucial gradient path:

$$\begin{aligned}&\text{heatmap logits} \to \text{softargmax waveform} \to \text{signal-domain loss}\\ &\to \text{gradient into the heatmap predictor.}\end{aligned}$$

7.5 Conversion from row index to millivolts

Once the expected row is computed, ECG-Net converts geometry into physiology. Let y_{mid} be the panel midpoint and let ppmv denote pixels per millivolt. Then the predicted amplitude is:

$$\hat{s}_{\text{mv}}(x) = \frac{y_{\text{mid}} - \hat{y}_{\text{soft}}(x)}{\text{ppmv}}.$$

This makes the output physically interpretable and allows the loss to be applied directly in calibrated units.

7.6 Why temperature 10.0?

The temperature controls sharpness:

- lower temperature: softer, blurrier row estimate,
- higher temperature: closer to hard selection but with less stable gradients.

A value of 10.0 is a reasonable middle ground: sharp enough to track a thin trace, but still differentiable enough for stable training.

8 Loss Design: Mathematical Form and Intuition

8.1 Why a composite loss is necessary

ECG-Net is not solving a single objective. It must:

- localize the trace,
- preserve overlap and continuity,
- recover correct amplitudes,
- suppress jitter,
- remain tolerant to slight horizontal misalignment.

No single off-the-shelf loss captures all of that, so the design uses multiple terms.

8.2 Notation

Let:

- z_{yx} be the predicted heatmap logit,
- h_{yx} be the target heatmap value,
- $\hat{s}_t$ be the predicted waveform value at column t,
- s_t be the target waveform value,
- T be the number of columns.

8.3 Heatmap loss: BCEWithLogits + Dice

8.3.1 BCE term

The BCEWithLogits term is:

$$L_{\text{BCE}} = -\frac{1}{HW} \sum_{y,x} \Big[\omega \, h_{yx} \log \sigma(z_{yx}) + (1 - h_{yx}) \log(1 - \sigma(z_{yx})) \Big],$$

where the positive-class weight is:

$$\texttt{POS_WEIGHT} = 6.0.$$

Plain-language meaning. The ECG trace occupies only a tiny fraction of pixels. If foreground and background were weighted equally, the model could minimize loss by predicting background almost everywhere. The positive weight prevents that collapse.

8.3.2 Dice term

The Dice loss is:

$$L_{\text{Dice}} = 1 - \frac{2 \sum_{y,x} p_{yx} h_{yx} + \varepsilon}{\sum_{y,x} p_{yx} + \sum_{y,x} h_{yx} + \varepsilon},$$

where $p_{yx} = \sigma(z_{yx})$.

Plain-language meaning. BCE is pixelwise. Dice asks whether the predicted trace shape overlaps the target shape overall. This is especially useful for thin foreground objects that can fragment.

8.3.3 Combined image-space loss

The effective heatmap loss is:

$$L_{\text{heat}} = L_{\text{BCE}} + 0.7 \, L_{\text{Dice}}.$$

8.4 Waveform L1 loss

The signal-domain reconstruction term is:

$$L_{\text{wave}} = \frac{1}{T} \sum_{t=1}^{T} |\hat{s}_t - s_t|.$$

Plain-language meaning. This penalizes numerical mismatch in physiological units. The trace should not only look right; it should decode to the correct waveform amplitude.

8.5 Energy-aware weighting

The implementation computes signal energy:

$$E = \frac{1}{T}\sum_{t=1}^{T} s_t^2,$$

and uses the weight

$$\alpha = \frac{E}{E+K}, \qquad K = \texttt{ENERGY_K} = 2 \times 10^{-4}.$$

So the weighted waveform term is:

$$L_{\text{wave,w}} = \alpha\, L_{\text{wave}}.$$

Why this helps. Very low-energy signals can be dominated by flat baseline regions, making raw L1 less informative. Energy weighting reduces the influence of those weak-signal cases.

8.6 First-derivative smoothness term

The first-difference penalty is:

$$L_{D1} = \frac{1}{T-1}\sum_{t=2}^{T} |\hat{s}_t - \hat{s}_{t-1}|.$$

Weight:

$$\texttt{SMOOTH_D1_W} = 0.02.$$

Meaning. This discourages high-frequency jitter between adjacent columns.

8.7 Second-derivative smoothness term

The second-difference penalty is:

$$L_{D2} = \frac{1}{T-2}\sum_{t=3}^{T} |\hat{s}_t - 2\hat{s}_{t-1} + \hat{s}_{t-2}|.$$

Weight:

$$\texttt{SMOOTH_D2_W} = 0.01.$$

Meaning. This penalizes unnatural zig-zag curvature and staircase artifacts.

8.8 Shift-tolerant alignment loss

In phase 2, ECG-Net introduces a soft minimum over a small set of horizontal shifts. If E_δ is the mean absolute error after shift δ, then:

$$L_{\text{shift}} = -\tau \log \sum_{\delta \in \Delta} \exp\left(-\frac{E_\delta}{\tau}\right),$$

with:

$$\texttt{SHIFT_TAU} = 0.035, \qquad \texttt{SHIFT_MAX_FRAC} = 0.010, \qquad \texttt{SHIFT_LOSS_W} = 0.55.$$

Meaning. If a prediction is nearly correct but shifted slightly left or right because of crop ambiguity, the model should not be punished as if it were morphologically wrong. This term gives a small alignment tolerance without needing full dynamic time warping.

8.9 Total objective by phase

Phase 1 uses:

$$L^{(1)} = L_{\text{heat}} + 0.22\, L_{\text{wave,w}} + 0.02\, L_{D1} + 0.01\, L_{D2}.$$

Phase 2 uses:

$$L^{(2)} = L_{\text{heat}} + 0.22\, L_{\text{wave,w}} + 0.55\, L_{\text{shift}} + 0.02\, L_{D1} + 0.01\, L_{D2}.$$

8.10 Amateur-friendly summary of the losses

- **BCE**: find trace pixels.
- **Dice**: keep the whole trace overlapping the target.
- **Wave L1**: recover the right waveform values.
- **D1**: remove jitter.
- **D2**: remove zig-zag curvature noise.
- **Shift loss**: do not overpunish tiny horizontal offsets.

9 Training Curriculum and Optimization Strategy

9.1 Two-phase training logic

The training is deliberately staged.

Phase 1. The model learns stable localization and basic signal reconstruction.

Phase 2. After epoch 10, the learning rate is reduced and the shift-tolerant term is activated. The phase-2 learning-rate multiplier is:

$$\texttt{PHASE2_LR_MULT} = 0.35.$$

This curriculum is good practice. If alignment flexibility were introduced too early, the model could exploit shift tolerance before learning good localization.

9.2 Optimizer and training hyperparameters

The main optimization settings are:

- optimizer: AdamW,
- learning rate: 3×10^{-4},
- weight decay: 10^{-2},
- batch size: 12,
- gradient clip: 5.0,
- epochs: 25.

9.3 Augmentation policy

The augmentation policy is intentionally light:

- random brightness/contrast with probability 0.50,
- Gaussian blur with probability 0.15.

This simulates contrast variability, scanner differences, and slight softness, but does not aggressively model large rotations or perspective distortion.

10 Metrics: What to Measure and Why

10.1 Primary metric: best-shift SNR

The most important validation metric is best-shift signal-to-noise ratio in decibels:

$$\mathrm{SNR}_{\mathrm{dB}} = 10 \log_{10} \left(\frac{\sum_t s_t^2}{\sum_t (s_t - \hat{s}_t)^2} \right),$$

with a small shift allowance during comparison.

This is the right checkpointing metric because the real goal is signal fidelity after digitization, not merely mask overlap.

The implementation also skips low-energy targets using:

$$\texttt{SNR_SKIP_ENERGY_THR} = 10^{-5}.$$

10.2 Secondary metric: trace_acc@2px

This metric measures the fraction of columns whose predicted row is within 2 pixels of the target row.

It is a geometric metric rather than a physiological one. It is useful, but it should not dominate checkpoint selection because a visually close trace can still decode to a poor waveform.

10.3 Why not optimize for Dice alone?

Dice is necessary but insufficient. A slightly thickened or shifted trace may still have decent overlap while yielding poor reconstructed amplitude. ECG-Net therefore prioritizes signal-level fidelity.

10.4 Recommended operating bands

The following are practical target ranges for this task class:

Metric	Suggested interpretation
Best-shift SNR > 20 dB	Excellent digitization quality
Best-shift SNR 15–20 dB	Strong and deployment-worthy
Best-shift SNR 10–15 dB	Usable but not robust
Best-shift SNR < 10 dB	Weak reconstruction quality
trace_acc@2px > 0.95	Excellent localization
trace_acc@2px 0.90–0.95	Strong localization
trace_acc@2px < 0.80	Too many geometric misses

These bands should not be treated as universal laws, but they are sensible engineering targets for a system of this type.

11 How ECG-Net Differs from Standard OCR

A standard OCR system is built for discrete symbol recognition. It detects text regions and decodes characters or token sequences.

ECG-Net is different in every important way:

- the target is a continuous analog waveform, not text,
- the output is a trace heatmap and calibrated signal, not character tokens,
- the geometry is column-wise curve localization, not text-line recognition,
- the loss is driven by both segmentation-style and signal-style supervision.

The simplest explanation is:

> OCR reads letters. ECG-Net reads a curve.

12 Alternatives and When They Can Outperform ECG-Net

12.1 Direct image-to-waveform regression

A CNN or transformer could map an image directly to a waveform vector.

When it can win: when the input layout is extremely standardized, crops are clean, and there is enough paired data to learn the full mapping end-to-end.

Why ECG-Net is stronger here: ECG-Net is more data-efficient and easier to debug because it isolates localization as an explicit subproblem.

12.2 Pure segmentation plus non-differentiable vectorization

Another approach is to train a segmentation model, then use dynamic programming, skeleton tracing, or handcrafted post-processing to extract the waveform.

When it can win: when deterministic extraction is very reliable and you care most about sharp trace geometry.

Why ECG-Net chose softargmax: softargmax allows the signal loss to directly train the heatmap predictor.

12.3 Classical computer vision pipeline

A fully handcrafted pipeline might use thresholding, morphology, Hough transforms, and contour tracing.

When it can win: on very clean, highly standardized scans with limited variation and little training data.

Why ECG-Net is preferred for generality: it is more robust to real-world variation and can learn beyond manually specified rules.

12.4 Different backbones

Possible alternatives include plain U-Net, HRNet, DeepLabV3+, or transformer-based decoders.

When they can win:

- HRNet: if ultra-fine localization dominates,
- DeepLabV3+: if broader semantic context matters more,
- transformer decoders: if data scale and appearance diversity are substantially larger.

Why EfficientNet-B0 + U-Net is reasonable: it is compact, practical, and well-matched to a thin-structure task.

12.5 Alternative losses

Other losses could include focal loss, Tversky loss, Huber loss, DTW-style losses, correlation loss, or frequency-domain losses.

When they can win:

- focal or Tversky: if false negatives become more critical,

- Huber: if outlier robustness matters,
- DTW: if local temporal warping is common,
- correlation or spectral losses: if morphology matters more than pointwise amplitude.

Why the current loss stack is good: it is minimal, interpretable, and closely tied to the specific failure modes of ECG digitization.

13 Senior-Level Architecture Defense

13.1 Why record-level splitting is the only honest evaluation choice

Because the record, not the panel, is the true source of correlated variation. Panel-level splitting leaks acquisition style and page-level artifacts into validation.

13.2 Why the trace-enhancement channel is not a hack

It is an engineered inductive bias. Since the encoder is not pretrained, it is entirely appropriate to give the model a domain-specific cue map that emphasizes thin dark curves.

13.3 Why the soft Gaussian heatmap is a probabilistic target

The target intentionally encodes uncertainty and thickness. It is not merely a blurred label; it is a smoother and more realistic representation of where the trace could plausibly be in a scanned image.

13.4 Why softargmax is the architectural hinge

Softargmax is the bridge that makes ECG-Net a vision-to-signal model instead of a segmentation model with a detached post-processing stage.

13.5 Why checkpoint by SNR instead of raw loss

Because downstream usefulness is waveform fidelity. Loss is only a training surrogate. SNR is closer to the actual project objective.

14 Interview Q&A Set for ECG-Net

14.1 Q1. Why record-level splitting rather than panel-level splitting?

Because panels from the same record are correlated in appearance and acquisition artifacts. Splitting at the panel level leaks record-specific information and inflates validation quality.

14.2 Q2. Why use a handcrafted trace-enhancement channel in a deep model?

Because the problem contains strong low-level priors: thin dark curves on structured paper. The handcrafted channel improves sample efficiency and optimization, especially with a non-pretrained encoder.

14.3 Q3. Why a soft Gaussian heatmap instead of a hard line target?

Because hard line targets are too sparse and brittle. Gaussian bands provide denser gradients, tolerate slight projection ambiguity, and better model real trace thickness.

14.4 Q4. What is Conv-BN-SiLU doing for you?

Convolution learns local patterns, BatchNorm stabilizes optimization, and SiLU provides smooth nonlinearity. The combination is effective for fine-grained visual reconstruction.

14.5 Q5. Why softargmax?

It approximates argmax while remaining differentiable, allowing signal-domain supervision to shape the heatmap logits directly.

14.6 Q6. How is this different from OCR?

OCR recognizes discrete symbols. ECG-Net reconstructs a continuous curve and calibrates it into a physiological time series.

14.7 Q7. Why these losses?

Because each loss suppresses a different failure mode: BCE for class imbalance, Dice for overlap, waveform L1 for amplitude accuracy, D1 and D2 for smoothness, and shift loss for alignment ambiguity.

14.8 Q8. When would direct waveform regression outperform this design?

When layout variability is minimal, crops are extremely clean, and training data is sufficiently abundant for direct end-to-end mapping.

14.9 Q9. Why not train with DTW directly?

DTW is heavier and can blur the distinction between alignment robustness and true geometric accuracy. ECG-Net uses a smaller shift-tolerant loss to gain alignment robustness without that complexity.

14.10 Q10. What is the biggest limitation of the current system?

The system still relies on a partly deterministic final extraction philosophy and uses relatively light augmentation, so robustness to highly distorted phone captures or unusual page geometries is still a natural improvement area.

15 Compact Hyperparameter Summary

Setting	Value
Input resolution	256×512
Input channels	4 (RGB + trace channel)
Backbone	`tf_efficientnet_b0`
Pretrained encoder	False
Target Gaussian sigma	2.0 px
Softargmax temperature	10.0
Positive class weight	6.0
Wave L1 weight	0.22
D1 smoothness weight	0.02
D2 smoothness weight	0.01
Shift loss weight	0.55
Shift tau	0.035
Max shift fraction	0.010
Optimizer	AdamW
Learning rate	3×10^{-4}
Weight decay	10^{-2}
Batch size	12
Gradient clip	5.0
Validation fraction	0.10
Phase 1 epochs	10
Total epochs	25
Phase 2 LR multiplier	0.35

16 Hyperparameter Roles and Why These Values Make Sense

This section expands the compact table above and explains the role of each major hyperparameter in system behavior, optimization stability, and final waveform fidelity. The important principle is that the values are not arbitrary. Each one reflects a trade-off between localization sharpness, numerical stability, computational cost, and robustness to scanned ECG artifacts.

16.1 Geometry and representation hyperparameters

Input resolution: 256×512**.** The panel height controls vertical localization precision, while the width controls horizontal waveform sampling density. A height of 256 keeps the vertical regression problem fine enough for thin traces without making the heatmap prohibitively expensive. A width of 512 is especially useful because the decoder later produces one waveform value per column. If the width were much smaller, sharp ECG morphology would be under-sampled; if it were much larger, training cost and memory usage would increase without a proportionate gain in accuracy.

Input channels: 4. The extra channel is the trace-enhancement prior. Using 4 channels instead of 3 slightly increases encoder complexity, but it gives the first layer access to a domain-specific map that emphasizes likely trace pixels. This is particularly justified because the encoder is trained from scratch rather than initialized from ImageNet weights.

Backbone: `tf_efficientnet_b0`. EfficientNet-B0 is the smallest member of the family that still gives a strong multi-scale feature pyramid. The choice balances three needs: enough semantic depth to distinguish trace from structured clutter, enough efficiency to train on moderately sized hardware, and enough simplicity to keep the decoder lightweight. A larger EfficientNet could improve representation power, but would raise cost and overfitting risk.

Pretrained encoder: False. This is not merely a switch; it changes the optimization regime. With a non-pretrained encoder, the model must learn low-level ECG-specific filters from scratch. That makes the handcrafted trace channel more valuable. The likely reasoning is that the input distribution and channel count differ materially from standard ImageNet settings, so a from-scratch encoder avoids awkward channel inflation and distribution mismatch. The trade-off is slower early training and greater dependence on the inductive biases built into preprocessing and targets.

16.2 Trace-enhancement hyperparameters

Trace black-hat fraction: $1/48$**.** The morphology kernel is scaled relative to image size instead of being fixed in pixels. This is good practice because the usefulness of black-hat depends on the visual scale of the trace relative to the background. A much smaller fraction would respond too locally and fail to separate the trace from nearby clutter; a much larger fraction would begin to absorb broader background patterns and weaken thin-curve selectivity.

Fusion weights: 0.70 black-hat and 0.30 edge response. These weights encode an explicit belief: dark curvilinear foreground is a more trustworthy prior than generic boundaries. Black-hat is therefore given the larger share. If the edge component dominated, grid lines, borders, and text would receive too much emphasis. If the black-hat component dominated too strongly, some contour sharpness would be lost. The chosen mixture keeps the channel trace-focused but still contour-aware.

16.3 Target and differentiable decoding hyperparameters

Gaussian sigma: 2.0 pixels. This hyperparameter controls how thick or soft the target trace band is. A sigma that is too small makes the target nearly binary, which leads to sparse gradients

and brittle optimization. A sigma that is too large allows the network to predict blurry, spatially imprecise traces without enough penalty. A value of 2.0 pixels is a good compromise for a 256-pixel-high panel: it is narrow enough to preserve thin-structure supervision, but wide enough to provide dense gradients near the correct row.

Softargmax temperature: 10.0. Softargmax uses a softmax distribution over height and converts it to an expected row index. The temperature determines how close this operation behaves to a hard argmax. A lower value spreads mass across many rows and produces overly smooth row estimates. A much higher value makes the distribution extremely sharp and can reduce gradient quality. A value of 10.0 is sharp enough to behave like a trace picker while still preserving differentiability for signal-domain learning.

16.4 Class imbalance and reconstruction-loss hyperparameters

Positive class weight: 6.0. The ECG trace occupies only a small part of the heatmap. Without class reweighting, BCE would be dominated by easy background pixels. Setting the positive weight to 6.0 increases the penalty for missing true trace regions and keeps the model from converging to an over-backgrounded solution. The value is large enough to counter imbalance, but not so large that the model starts hallucinating foreground everywhere.

Wave L1 weight: 0.22. This term determines how strongly the decoded waveform influences the model relative to image-space heatmap supervision. If it were too small, the system would behave like a segmentation model with only weak signal awareness. If it were too large, the heatmap could become unstable early in training because the waveform loss is downstream of the decoder and initially noisy. A moderate value like 0.22 says: localization remains primary, but numerical waveform correctness is important enough to shape the learned representation.

Energy stabilizer: 2×10^{-4}. The waveform L1 term is energy-weighted to reduce the influence of very low-energy or near-flat segments. This constant prevents the denominator from becoming too small and keeps the weighting numerically stable. If the stabilizer were much smaller, low-energy examples could still create erratic scaling; if much larger, the energy weighting would lose sensitivity.

16.5 Smoothness and alignment hyperparameters

First-derivative smoothness weight: 0.02. This penalizes abrupt step-to-step changes in the recovered signal. It is intentionally small because ECG signals must retain sharp physiological morphology such as QRS transitions. The goal is not to flatten the waveform, but to suppress pixel-column jitter introduced by noisy heatmaps.

Second-derivative smoothness weight: 0.01. This term discourages high-frequency zig-zag curvature and staircase artifacts. It is smaller than the first-derivative penalty because second-order smoothing is more aggressive. If it were too large, clinically meaningful curvature would be oversmoothed.

Shift loss weight: 0.55. Once the model already localizes well, small horizontal alignment mismatches become a major nuisance. The shift-tolerant loss is therefore weighted substantially in phase 2. A weight of 0.55 makes it strong enough to matter, but not so dominant that the model ignores exact localization. The training curriculum is important here: this term is introduced after basic trace learning has already stabilized.

Shift temperature: 0.035. The shift loss uses a soft minimum over several alignment candidates. This temperature determines how close the soft minimum is to the true minimum. A smaller value makes the operation behave more like a hard minimum, while a larger value averages too much across poor alignments. A value of 0.035 is sharp enough to emphasize the best nearby alignment without making optimization discontinuous.

Maximum shift fraction: 0.010. This controls how far the model is allowed to compensate for horizontal misalignment. Setting it to 1% of width keeps the tolerance local. That is exactly the intended use case: correct for tiny rasterization or crop offsets, not large registration mistakes. If this value were much larger, the model could hide genuine extraction errors inside an overly forgiving alignment window.

16.6 Optimization hyperparameters

Optimizer: AdamW. AdamW is a good fit because the network mixes convolutional layers, batch normalization, and a multi-term objective with different gradient scales. AdamW provides adaptive learning rates while keeping weight decay decoupled from the gradient update. That usually behaves better than plain Adam when one wants cleaner regularization.

Learning rate: 3×10^{-4}. This is a standard moderately aggressive learning rate for AdamW in medium-scale vision models. It is large enough to train efficiently from scratch, but not so large that the heatmap and waveform objectives destabilize each other. With a non-pretrained encoder, this value helps the model learn useful low-level filters quickly.

Weight decay: 10^{-2}. A weight decay of 0.01 is a common regularization strength for AdamW. It discourages overly large weights, improves generalization, and helps keep the from-scratch encoder from overfitting to record-specific artifacts. If it were much smaller, the model might memorize nuisance textures more easily; if much larger, it could underfit thin-trace details.

Batch size: 12. The batch size is governed by a compromise between gradient stability and GPU memory footprint. Heatmap supervision at 256×512 resolution is not cheap, especially with skip connections and mixed losses. A batch size of 12 is large enough to make BatchNorm useful and to stabilize updates, but small enough to remain practical.

Gradient clipping: 5.0. The network combines segmentation-style and sequence-style supervision. During early training, decoded waveform gradients can spike if the heatmap is poor. Gradient clipping at 5.0 prevents rare unstable steps from corrupting training dynamics. This is a protective hyperparameter rather than a performance hyperparameter.

16.7 Training-schedule hyperparameters

Validation fraction: 0.10. Ten percent validation is a pragmatic setting: enough to monitor generalization and checkpoint selection, but not so large that it needlessly reduces the training set. Because splitting is done at the record level, the main concern is independence rather than raw count alone.

Phase 1 epochs: 10. The first stage is long enough for the model to learn trace localization, stabilize the heatmap predictor, and produce a meaningful waveform estimate. It is intentionally not too long because the more permissive phase-2 objective should take over once the base geometry is learned.

Total epochs: 25. This suggests the project expects the model to converge in a relatively compact training window. A total of 25 epochs is sufficient when the task is tightly structured and the supervision is dense. Much longer training would increase the risk of overfitting record-specific appearance.

Phase-2 learning-rate multiplier: 0.35. When the shift-tolerant objective is activated, the learning rate is reduced to 35% of its original value. This is sensible because phase 2 is more about refinement than discovery. The model already knows roughly where the trace is; the lower learning rate prevents the refined objective from destabilizing previously learned localization.

16.8 How to interpret these hyperparameters as a system

The hyperparameters are best understood as a coordinated package rather than isolated numbers. The geometry choices make the heatmap representation viable. The trace-channel settings provide inductive bias for a from-scratch encoder. The Gaussian and softargmax settings make the image-to-signal bridge trainable. The loss weights keep localization primary while still enforcing waveform fidelity. The optimization schedule then introduces alignment robustness only after the base extraction problem is mostly solved. In that sense, the hyperparameters encode the project philosophy: first learn where the ECG trace is, then learn how to make the extracted waveform numerically robust.

17 Final Technical Summary

The strongest aspect of ECG-Net is not any one individual layer. The strength comes from the *coupling* of design choices:

- domain-informed preprocessing,
- a thin-structure heatmap representation,
- a compact multi-scale encoder-decoder,
- a differentiable bridge from 2D heatmap to 1D signal,
- loss terms that each address a specific failure mode,

- checkpointing by the actual downstream metric.

That is why ECG-Net reads as more than "a U-Net on ECGs." It is a carefully shaped vision-to-signal system whose architecture, targets, and losses are all aligned to the real problem of ECG digitization.